

# **JOB PORTAL MANAGEMENT SYSTEM**

*School of Computing*

**Bachelor of Technology  
in  
Computer Science & Engineering**

**By**

**K.Karthikeya(VTU25637)**

*10211CS224*

*Full Stack Application Development*

*Slot : E2*



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING  
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF  
SCIENCE AND TECHNOLOGY**

**(Deemed to be University Estd u/s 3 of UGC Act, 1956)**

**CHENNAI 600 062, TAMILNADU, INDIA**

**May, 2026**

# **BONAFIDE CERTIFICATE**

It is certified that the work contained in the Full Stack Application Development report titled "JOB PORTAL MANAGEMENT SYSTEM" by K.Karthikeya(VTU25637) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

**Signature of Faculty**

**Dr. Chidambaranatham C M.**

**Associate Professor**

**Computer Science & Engineering**

**School of Computing**

**Vel Tech Rangarajan Dr.Sagunthala R&D**

**Institute of Science and Technology**

**April, 2026**

**Signature of Head of the Department**

**Dr. M.Kavitha**

**Professor & Head**

**Computer Science & Engineering**

**School of Computing**

**Vel Tech Rangarajan Dr.Sagunthala R&D**

**Institute of Science and Technology**

**April, 2026**

# DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(K.Karthikeya)

Date:06/05/2026

# ABSTRACT

The Job Portal Management System is a full-stack web application designed to bridge the gap between job seekers and employers through a secure, efficient, and feature-rich digital recruitment platform. In today's competitive job market, there is a critical need for a unified system that enables employers to post and manage job openings while simultaneously allowing candidates to discover opportunities, apply seamlessly, and track their application status in real time.

This system is built using React 18 (Vite) on the frontend and Spring Boot 3.x (Java 21) on the backend, integrated with a MySQL relational database. The application enforces security through JSON Web Tokens (JWT) for stateless authentication, ensuring that employer and job seeker sessions are isolated and protected. Employers can register, post both remote and on-site job listings, review applicants, download candidate resumes, and update application statuses such as Shortlisted or Rejected. Job seekers can create profiles with uploaded resumes, browse the active job board, apply with a single click.

**Keywords:** Job Portal, Full Stack Development, React, Spring Boot, JWT Authentication, MySQL, REST API, Recruitment System, Resume Upload, Notification Engine.

# LIST OF FIGURES

|     |                                                                     |    |
|-----|---------------------------------------------------------------------|----|
| 1.1 | Framework Diagram . . . . .                                         | 4  |
| 3.1 | Backend Implementation . . . . .                                    | 10 |
| 4.1 | Project Architecture . . . . .                                      | 14 |
| 4.2 | WorkFlow . . . . .                                                  | 15 |
| 4.3 | DataFlow Daigram . . . . .                                          | 16 |
| 5.1 | Registration and Login Page . . . . .                               | 20 |
| 5.2 | Employer Dashboard – Job Listings and Applicant Count Metrics .     | 21 |
| 5.3 | Job Seeker – Job Board Listings and Application Status Tracker . .  | 21 |
| 5.4 | Notification Bell Showing Unread Status Alerts for the Job Seeker . | 22 |
| 5.5 | Notification Bell Showing Unread Status Alerts for the Job Seeker . | 23 |
| 5.6 | Type Location or Title to Search Job . . . . .                      | 23 |
| 5.7 | JobSeeker Profile . . . . .                                         | 24 |

# LIST OF TABLES

|     |                                                                                              |    |
|-----|----------------------------------------------------------------------------------------------|----|
| 2.1 | Comparison of Existing Job Portal Applications . . . . .                                     | 6  |
| 5.1 | System Modules and Their Features . . . . .                                                  | 18 |
| 5.2 | Feature Implementation Status . . . . .                                                      | 19 |
| 5.3 | Average API Response Times . . . . .                                                         | 25 |
| 7.1 | Mapping of Job Portal Management System to Complex Engineering<br>Problem (CEP) . . . . .    | 29 |
| 7.2 | Mapping of Job Portal Management System to Complex Engineering<br>Activities (CEA) . . . . . | 30 |
| 7.3 | SDG Mapping for the Job Portal Management System . . . . .                                   | 31 |

# LIST OF ACRONYMS AND ABBREVIATIONS

|      |                                   |
|------|-----------------------------------|
| API  | Application Programming Interface |
| CSS  | Cascading Style Sheets            |
| HTML | HyperText Markup Language         |
| HTTP | HyperText Transfer Protocol       |
| JPA  | Java Persistence API              |
| JSON | JavaScript Object Notation        |
| JWT  | JSON Web Token                    |
| MVC  | Model-View-Controller             |
| ORM  | Object Relational Mapping         |
| REST | Representational State Transfer   |
| SPA  | Single Page Application           |
| SQL  | Structured Query Language         |
| UI   | User Interface                    |

# TABLE OF CONTENTS

|                                                  | Page.No    |
|--------------------------------------------------|------------|
| <b>ABSTRACT</b>                                  | <b>iii</b> |
| <b>LIST OF FIGURES</b>                           | <b>iv</b>  |
| <b>LIST OF TABLES</b>                            | <b>v</b>   |
| <b>LIST OF ACRONYMS AND ABBREVIATIONS</b>        | <b>vi</b>  |
| <b>1 INTRODUCTION</b>                            | <b>1</b>   |
| 1.1 Introduction . . . . .                       | 1          |
| 1.2 Aim of the Project . . . . .                 | 2          |
| 1.3 Scope of the Project . . . . .               | 3          |
| 1.4 Framework . . . . .                          | 3          |
| <b>2 SURVEY OF SIMILAR APPLICATION IN MARKET</b> | <b>5</b>   |
| <b>3 FRAMEWORK DESCRIPTION</b>                   | <b>7</b>   |
| 3.1 Frontend Implementation . . . . .            | 7          |
| 3.2 Backend Implementation . . . . .             | 9          |
| 3.3 Database Implementation . . . . .            | 11         |
| <b>4 METHODOLOGIES</b>                           | <b>13</b>  |

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| 4.1      | Project Architecture . . . . .                                   | 13        |
| 4.2      | DataFlow Diagram . . . . .                                       | 14        |
| 4.3      | Module 1: Authentication Module . . . . .                        | 15        |
| 4.4      | Module 2: Job and Application Management Module . . . . .        | 16        |
| 4.5      | Module 3: Profile and Notification Module . . . . .              | 17        |
| <b>5</b> | <b>RESULTS AND DISCUSSIONS</b>                                   | <b>18</b> |
| 5.1      | System Modules and Features . . . . .                            | 18        |
| 5.2      | User Interface Screenshots Walkthrough . . . . .                 | 20        |
| 5.3      | Performance Analysis . . . . .                                   | 24        |
| <b>6</b> | <b>CONCLUSION AND FUTURE ENHANCEMENTS</b>                        | <b>27</b> |
| 6.1      | Conclusion . . . . .                                             | 27        |
| 6.2      | Future Enhancements . . . . .                                    | 28        |
| <b>7</b> | <b>ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG</b>            | <b>29</b> |
| 7.1      | Mapping of the Project to Complex Engineering Problem (CEP) . .  | 29        |
| 7.2      | Mapping of the Project to Complex Engineering Activities (CEA) . | 30        |
| 7.3      | SDG Mapping . . . . .                                            | 31        |
|          | <b>References</b>                                                | <b>31</b> |

# Chapter 1

## INTRODUCTION

### 1.1 Introduction

The rapid growth of the internet and digital technologies has fundamentally transformed how businesses recruit talent and how individuals search for employment opportunities. Traditional job hunting methods — newspaper advertisements, walk-in interviews, and physical resume submissions — have been largely replaced by on-line platforms that offer speed, reach, and convenience. However, many existing job portals suffer from performance issues, lack of security, and poor user experience, particularly for the candidate side of the workflow.

The **Job Portal Management System** is a comprehensive full-stack web application developed to address these shortcomings. It serves as a centralised digital ecosystem connecting two primary user roles: **Employers** and **Job Seekers**. Employers can register on the platform, post detailed job listings (both remote and on-site), review candidate applications, download resumes, and update the recruitment status of each applicant. Job seekers, on the other hand, can create and maintain their profiles, upload their resumes, browse open positions, apply with a single click, and receive instant in-app notifications whenever their application status changes.

The system is built on a modern, industry-standard technology stack. The frontend is developed using **React 18** powered by **Vite**, delivering a fast and responsive Sin-

gle Page Application (SPA). The backend is implemented with **Spring Boot 3.x** on **Java 21**, exposing secure RESTful APIs. Authentication is enforced through **JSON Web Tokens (JWT)**, and data is persisted in a **MySQL** relational database managed through **Spring Data JPA** with Hibernate as the ORM layer.

## 1.2 Aim of the Project

The primary aim of this project is to design and implement a secure, scalable, and user-friendly Job Portal Management System that streamlines the entire recruitment lifecycle. Specifically, the project aims to:

- Provide employers with a reliable platform to post, manage, and track job listings efficiently.
- Enable job seekers to discover relevant job opportunities and apply without friction.
- Implement robust JWT-based authentication to ensure role-specific access control for employers and job seekers.
- Build a real-time notification system that alerts candidates instantly when an employer updates their application status.
- Develop a clean, intuitive user interface that minimises the learning curve for all user types.
- Demonstrate best practices in full-stack application development using React and Spring Boot.

### 1.3 Scope of the Project

The scope of the Job Portal Management System encompasses the following functional boundaries:

- **User Management:** Registration and login for both Employers and Job Seekers with JWT-secured sessions.
- **Job Listings:** Full CRUD (Create, Read, Update, Delete) operations on job postings by employers, including job type (remote/on-site), description, and requirements.
- **Application Management:** Job seekers can apply for open positions; employers can view all applicants per job posting and take actions (Shortlist or Reject).
- **Resume Handling:** Job seekers can upload resumes to their profiles; employers can download these resumes directly.
- **Notification Engine:** In-app notifications are generated whenever an employer changes a candidate's application status, with unread counts displayed on the seeker's dashboard.
- **Dashboard Metrics:** Employers see aggregated metrics (total active jobs, total applicants); seekers track their application statuses.

The project does not include payment processing, video interviewing, AI-based resume matching, or third-party email notifications in its current scope.

### 1.4 Framework

The application adopts a decoupled **client-server architecture** where the frontend and backend operate as independent services communicating through REST APIs.

The **Frontend** is a Single Page Application built with React. React's component-based architecture enables modular UI development, while Vite provides near-instant hot module replacement during development. Axios is used as the HTTP client for API calls, and React Router DOM handles client-side navigation.

The **Backend** is a RESTful API server built with Spring Boot 3.x. Spring Security handles authentication and authorisation with JWT tokens. Spring Data JPA with Hibernate acts as the ORM layer, abstracting SQL operations into Java objects. The backend runs on port 8081 and automatically manages the MySQL schema via Hibernate's DDL auto-generation.

The **Database** is MySQL, a widely adopted open-source relational database. It stores user accounts, employer profiles, job seeker profiles, job listings, applications, and notifications in normalised relational tables.

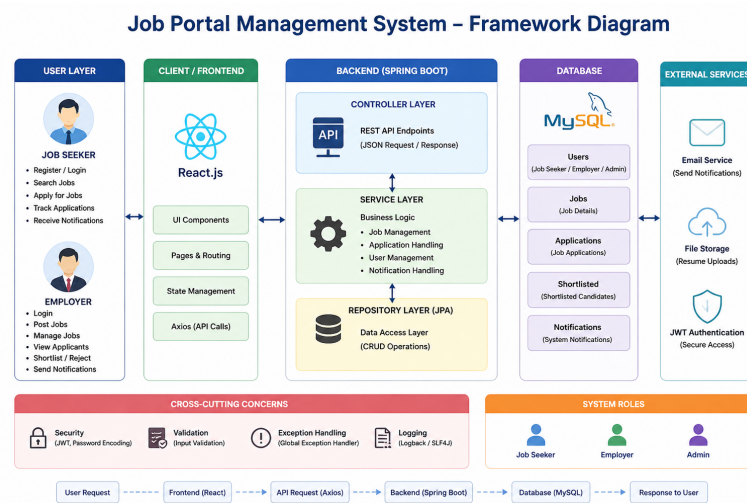


Figure 1.1: Framework Diagram

Figure 1.1 says about the framework diagram of the Job Portal Management System, illustrating the overall architecture and interaction between the frontend, backend, database, and external services, showing how data flows smoothly across components and how different technologies work together to deliver an efficient, scalable, and well-integrated system.

## Chapter 2

# SURVEY OF SIMILAR APPLICATION IN MARKET

A thorough analysis of existing job portal platforms in the market was conducted to identify their strengths, limitations, and the gaps that the proposed system aims to address.

**LinkedIn** is the world's largest professional networking and job portal platform. It combines job postings with professional profiles, recommendations, and a social networking layer. While powerful, LinkedIn is a general-purpose professional network; its job portal features are bundled with many unrelated social features, making it heavy for users who only need a pure recruitment workflow. Additionally, resume management and direct application tracking are less streamlined compared to dedicated ATS (Applicant Tracking Systems).

**Naukri.com** is India's leading job portal, offering a large database of job listings and candidates. It provides resume uploading, job alerts, and application tracking. However, the platform is cluttered with advertisements, and its user interface has become dated. Employers on Naukri must purchase credits to contact candidates, creating a barrier for small businesses.

**Indeed** is a global job aggregator that indexes listings from company websites and job boards. While its search capabilities are strong, Indeed lacks deep employer-

candidate lifecycle management. Employers have limited control over the application pipeline, and in-app notification systems for candidates are minimal.

**Internshala** focuses on internships and entry-level jobs, particularly relevant to students in India. It offers a clean UI but is restricted in scope and does not cater to mid-level or senior job seekers.

### Comparison Summary:

| Feature              | LinkedIn | Naukri  | Indeed  | Our System |
|----------------------|----------|---------|---------|------------|
| JWT Auth             | Yes      | Yes     | Yes     | Yes        |
| Resume Upload        | Yes      | Yes     | No      | Yes        |
| In-App Notifications | Partial  | No      | No      | Yes        |
| Status Tracking      | Partial  | Partial | No      | Yes        |
| Free for Employers   | No       | No      | Partial | Yes        |
| Lightweight UI       | No       | No      | Yes     | Yes        |

Table 2.1: Comparison of Existing Job Portal Applications

Table 2.1 shows the comparison of existing job portal applications based on various features such as authentication, resume upload, notifications, and system efficiency, highlighting that the proposed system provides more complete and user-friendly functionalities compared to platforms like LinkedIn, Naukri, and Indeed.

## Chapter 3

# FRAMEWORK DESCRIPTION

### 3.1 Frontend Implementation

#### 3.1.1 Environment Setup

The frontend environment is set up using Node.js and npm. Vite is used as the build tool and development server, chosen for its significantly faster startup and hot module replacement compared to traditional Create React App setups.

Prerequisites:

- Node.js v18 or above
- npm v9 or above

Installation steps:

Listing 3.1: Frontend Setup Commands

```
cd frontend
npm install
npm run dev
```

The development server starts at **http://localhost:5173**. The **vite.config.js** file configures a proxy to forward API requests to the Spring Boot backend at **http://localhost:8081**.

#### 3.1.2 Structure of the Project

The React project follows a feature-based folder structure:

### Listing 3.2: Frontend Folder Structure

```
frontend /
  src /
    components /      # Reusable UI components (Navbar, NotificationBell)
    pages /           # Page-level components (Login, Register, Dashboard)
      employer /      # Employer-specific pages (PostJob, ViewApplicants)
      seeker /        # Seeker-specific pages (JobBoard, MyApplications)
    api /             # Axios instance and API call functions
    context /         # Auth context for global state management
    App.jsx           # Root component with React Router routes
    main.jsx          # Vite entry point
  public /
  index.html
  vite.config.js
```

#### Key frontend dependencies:

- **react-router-dom**: Handles client-side routing and protected routes.
- **axios**: HTTP client for REST API communication; configured with a base URL and an interceptor that automatically attaches the JWT token to every outgoing request header.
- **lucide-react**: Provides lightweight SVG icons for the UI.

### 3.1.3 Execution Procedure

1. Navigate to the `frontend` directory.
2. Run `npm install` to install all dependencies listed in `package.json`.
3. Run `npm run dev` to start the Vite development server.
4. Access the application at `http://localhost:5173`.
5. The Axios interceptor reads the JWT token from `localStorage` and appends it as a Bearer token in the `Authorization` header for all API calls.
6. On login, the token and user role are stored in React context and `localStorage` for session persistence across page refreshes.

## 3.2 Backend Implementation

### 3.2.1 Environment Setup

The backend is a Maven-based Spring Boot project. It requires:

- Java Development Kit (JDK) 21
- Maven 3.8+ (or use the included Maven Wrapper `mvnw`)
- MySQL Server 8.x running locally

Listing 3.3: Backend Startup Command

```
cd backend
./mvnw spring-boot:run
```

The backend server starts on port 8081. The `application.properties` file configures the database connection, JWT secret, token expiry, and multipart file upload limits.

### 3.2.2 Structure of the Project

The Spring Boot project follows the standard layered MVC architecture:

Listing 3.4: Backend Package Structure

```
src/main/java/com/jobportal/
controller/      # REST controllers (AuthController, JobController, etc.)
service/         # Business logic layer
repository/      # Spring Data JPA repositories
model/           # JPA Entity classes (User, Job, Application, Notification)
dto/             # Data Transfer Objects for request/response
security/        # JWT utility, filter, and Spring Security config
config/          # CORS configuration
```

Key backend dependencies (in `pom.xml`):

- **spring-boot-starter-web**: Embeds Tomcat and enables REST controller support.
- **spring-boot-starter-security**: Provides the security filter chain.
- **spring-boot-starter-data-jpa**: Enables JPA repositories and Hibernate ORM.

- **jjwt**: Library for generating and validating JWT tokens.
- **mysql-connector-j**: JDBC driver for MySQL connectivity.

### 3.2.3 Execution Procedure

1. Ensure MySQL is running and the `job_portal` database is created.
2. Configure `src/main/resources/application.properties` with correct database credentials.
3. Run `./mvnw spring-boot:run` from the backend directory.
4. Spring Boot auto-creates all database tables via Hibernate DDL on first startup (`spring.jpa.hibernate.ddl-auto=update`).
5. The server is ready to accept HTTP requests at `http://localhost:8081/api/`.
6. All endpoints except `/api/auth/**` require a valid JWT Bearer token in the request header, enforced by the `JwtAuthenticationFilter`.

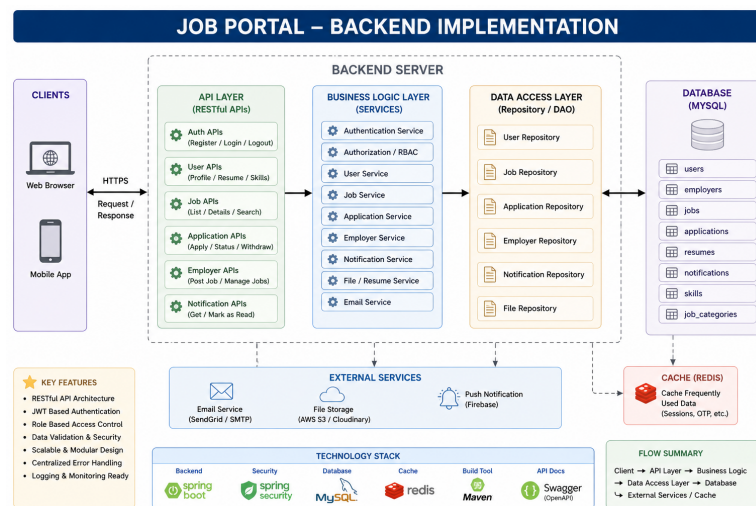


Figure 3.1: Backend Implementation

Figure 3.1 shows the backend implementation of the Job Portal Management System, illustrating the structure of the backend server including API layer, business logic layer, data access layer, and database, along with their interactions with clients

### 3.3 Database Implementation

#### 3.3.1 Database Configuration

The Job Portal Management System uses a relational database (MySQL) to store and manage application data efficiently. The database is designed to handle multiple entities such as users, employers, job listings, applications, resumes, and notifications. Proper normalization is applied to ensure data consistency and avoid redundancy.

The configuration includes secure database connectivity using environment variables for credentials such as database URL, username, and password. The backend establishes a connection using a database driver and ORM (Object Relational Mapping) framework, enabling smooth interaction between the application and database. Connection pooling is implemented to improve performance and handle multiple user requests efficiently.

The system defines separate tables for each entity with primary and foreign key relationships to maintain data integrity. For example, users and employers are linked to job postings and applications through unique identifiers. Indexing is applied on frequently searched fields like job title, location, and skills to optimize query performance.

#### 3.3.2 Schema Structure

The database schema is automatically generated by Hibernate based on the JPA entity annotations in the Spring Boot project.

The primary entities are:

- **users:** Stores credentials and role (EMPLOYER / JOB\_SEEKER) for all registered users.
- **employer\_profiles:** Stores employer-specific details linked to the users table.

- **seeker\_profiles:** Stores job seeker bio, skills, contact info, and resume file path.
- **jobs:** Stores all job listings posted by employers with title, description, type, and status.
- **applications:** A junction table linking seekers to jobs, storing application status (PENDING, SHORTLISTED, REJECTED).
- **notifications:** Stores in-app notification messages for seekers with a read/unread flag.

### 3.3.3 Tables created

- **Users:** Stores user information including name, email, encrypted password, role (Admin/User), and contact details.
- **Employer Profiles:** Contains employer-specific details such as company name, industry, and associated user ID.
- **Seeker Profiles:** Stores job seeker details including bio, skills, and resume path linked to the user.
- **Jobs:** Contains job-related information such as job title, description, job type, and status, posted by employers.
- **Applications:** Stores application details including job ID, seeker ID, application status, and applied date.
- **Notifications:** Maintains notification records such as messages, read status, and timestamps for users.

## Chapter 4

# METHODOLOGIES

### 4.1 Project Architecture

The Job Portal Management System adopts a **three-tier client-server architecture**:

- **Presentation Tier (Frontend):** The React SPA serves as the presentation layer. Users interact with the application through a browser. React Router manages navigation between pages without full-page reloads, while Axios handles asynchronous communication with the backend REST API. Custom CSS variables ensure a consistent theme across all components.
- **Application Tier (Backend):** The Spring Boot application acts as the application server. It exposes RESTful endpoints under `/api/` that the frontend consumes. Business logic — including application status updates, notification generation, and resume file storage — is handled in the service layer. The Spring Security filter chain intercepts all requests, validates the JWT token, and sets the authenticated user in the security context before the request reaches the controller.
- **Data Tier (Database):** MySQL stores all persistent data. Spring Data JPA repositories provide an abstraction over native SQL queries. Hibernate manages

entity relationships and translates Java object operations into optimised SQL statements.

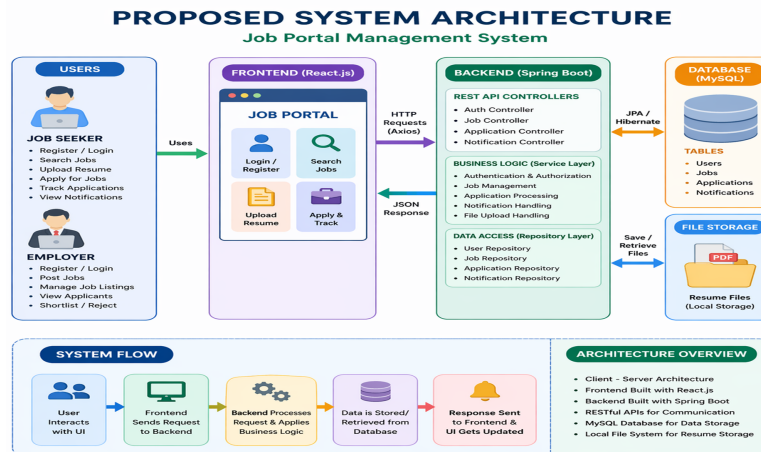


Figure 4.1: Project Architecture

Figure 4.1 shows the proposed system architecture of the Job Portal Management System, illustrating the interaction between users, frontend, backend services, and database, and how data flows through different layers to ensure efficient processing, secure communication, and seamless functionality of the application.

#### 4.1.1 WorkFlow

The Job Portal Management System enables seamless interaction between job seekers and employers through an organized workflow. The process begins when a user opens the application and either registers as a new user or logs in with existing credentials. The system performs authentication and role validation to direct users to the appropriate dashboard.

## 4.2 DataFlow Diagram

The data flow in the system is as follows:

1. A **Job Seeker** registers and logs in, receiving a JWT token.

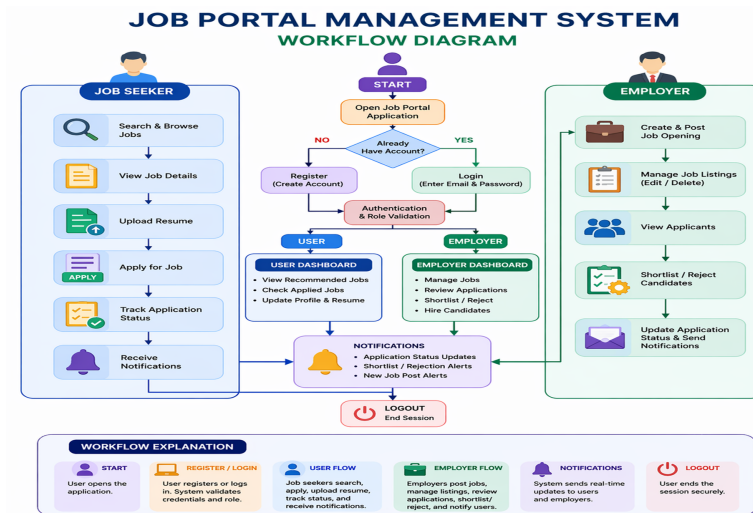


Figure 4.2: WorkFlow

- The seeker browses available jobs using `GET /api/jobs`, and active job listings are retrieved from the database.
- The seeker applies via `POST /api/applications`, creating a record in applications with status `PENDING`.
- An **Employer** logs in and views applications using `GET /api/jobs/{jobId}/applications`.
- The employer updates status via `PUT /api/applications/{id}/status`.
- The backend service generates a `Notification` record linked to the job seeker.
- The seeker dashboard fetches `GET /api/notifications`, displaying unread alerts.

### 4.3 Module 1: Authentication Module

The Authentication Module manages the complete user lifecycle for both Employers and Job Seekers.

**Registration:** Both user types register with their email, password, and role. A corresponding profile record (`employer_profile` or `seeker_profile`) is also created.

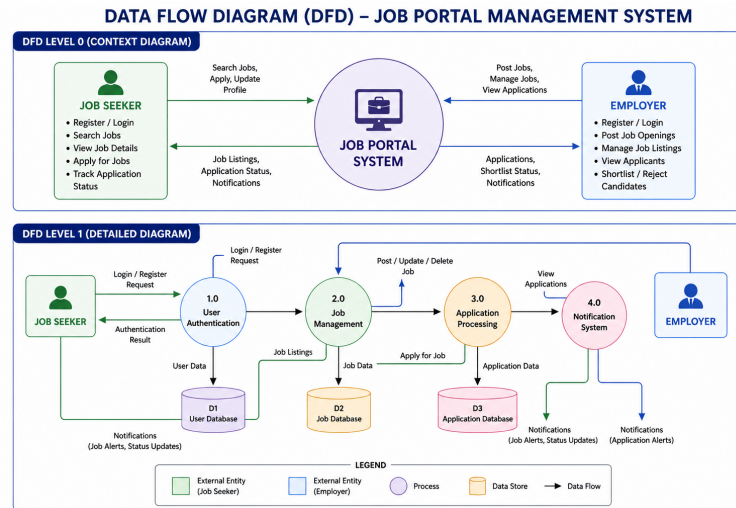


Figure 4.3: DataFlow Daigram

**Login:** On login, the backend verifies the BCrypt hash and generates a signed JWT token using a secret key with a configurable expiry (default: 24 hours). The token encodes the user ID and role as claims.

**Protected Routes:** On the frontend, React Router's private route wrappers check for a valid token in context. On the backend, the `JwtAuthenticationFilter` validates the token signature and expiry on every incoming request.

## 4.4 Module 2: Job and Application Management Module

**Employer Side:** The employer is provided with functionalities to manage job postings and applicants efficiently. Employers can create new job listings by providing details such as title, description, requirements, and job type (remote or on-site). They can also update or delete existing job postings as needed.

Additionally, employers can view all applicants for a specific job and download the resumes of job seekers. The system also allows employers to update the application status, such as shortlisted or rejected, enabling effective candidate management.

**Seeker Side:** Job seekers are provided with features to explore and apply for job opportunities. They can browse all active job listings and apply to suitable positions

easily. They can also view all submitted applications and track their current status, helping them stay informed throughout the recruitment process.

## 4.5 Module 3: Profile and Notification Module

**Profile Management:** Job seekers can update their profile information including bio, skills (stored as a comma-separated string), phone number, and LinkedIn URL via `PUT /api/seekers/profile`. They can also upload a resume file (PDF or DOCX) via `POST /api/seekers/resume`, which stores the file on the local server filesystem and records the file path in the database.

**Notification Engine:** Whenever an employer updates an application status to `SHORTLISTED` or `REJECTED`, the backend service automatically inserts a Notification entity into the database with a descriptive message and sets `is_read = false`. The seeker's dashboard polls `GET /api/notifications/unread-count` to display the badge on the notification bell icon. Clicking the bell marks all notifications as read via `PUT /api/notifications/mark-read`.

### Advantages of this Project

- a) Security:** JWT-based stateless authentication secures user sessions without server-side state, enabling easy horizontal scaling.
- b) Real-Time Feedback:** The in-app notification system gives candidates instant updates on their application status.
- c) Decoupled Architecture:** The React frontend and Spring Boot backend are fully separated, enabling independent development and scaling.

**Disadvantages** The system currently uses client-side polling for notifications, which may cause slight delays in real-time updates. Resume files are stored locally, but cloud storage solutions like AWS.

## Chapter 5

# RESULTS AND DISCUSSIONS

### 5.1 System Modules and Features

The Job Portal Management System is composed of five tightly integrated modules, each responsible for a distinct aspect of the recruitment workflow. The following table summarises the modules, their key features, and the technology components involved in their implementation.

| Module                | Key Features                                                                                                        | Technologies Used                                  |
|-----------------------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Authentication Module | User registration (Employer / Seeker), JWT login, BCrypt password hashing, role-based route protection              | Spring Security, JWT, React Context API            |
| Job Management Module | Post / Edit / Delete job listings, remote and on-site job types, active/inactive toggle, employer dashboard metrics | Spring Boot REST, JPA, React Router                |
| Application Module    | Single-click apply, status tracking (Pending / Shortlisted / Rejected), applicant list per job, resume download     | Spring Data JPA, Axios, MySQL                      |
| Profile Module        | Seeker bio and skills update, multipart resume file upload, employer company profile management                     | Spring Multipart, Local FS, React Forms            |
| Notification Module   | In-app notification bell, unread count badge, mark-all-read, status change alerts triggered by employer actions     | JPA Notification Entity, React State, REST Polling |

Table 5.1: System Modules and Their Features

Table 5.1 says about the System Modules and Their Features, detailing the core architectural components of the job portal such as Authentication, Job Management, and Application modules. It explicitly lists key features for each module—including role-based protection, resume file uploads, and status tracking—alongside the specific technologies used like Spring Boot, React, and MySQL.

Each module exposes dedicated REST endpoints and is backed by a corresponding Spring Boot service class that encapsulates the business logic. On the frontend, each module maps to a set of React page components and reusable UI components. The modular design ensures that changes to one module do not affect the behaviour of others, facilitating easier testing and future extension.

### Feature Completeness Summary:

| Feature                           | Status      |
|-----------------------------------|-------------|
| Employer Registration & Login     | Implemented |
| Job Seeker Registration & Login   | Implemented |
| JWT Token Authentication          | Implemented |
| Post / Edit / Delete Job Listings | Implemented |
| Browse Job Board (Seeker)         | Implemented |
| Single-Click Job Application      | Implemented |
| Applicant List per Job (Employer) | Implemented |
| Resume Upload (Seeker)            | Implemented |
| Application Status Update         | Implemented |
| In-App Notification Engine        | Implemented |
| Employer Dashboard Metrics        | Implemented |
| Seeker Profile Management         | Implemented |

Table 5.2: Feature Implementation Status

Table 5.2 says about Features, detailing the core architectural components of the job portal such as Authentication, Job Management, and Application modules. It explicitly lists key features for each module—including role-based protection, resume file uploads, and status tracking—alongside the specific technologies used like Spring Boot, React, and MySQL.

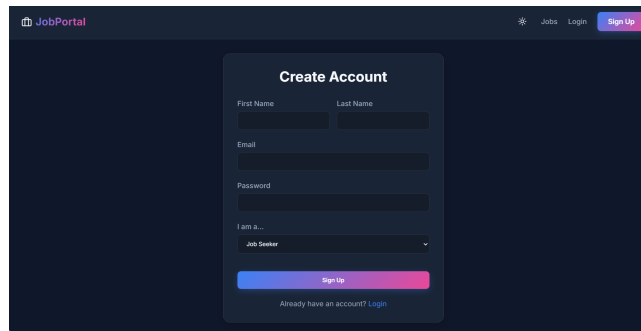


Figure 5.1: Registration and Login Page

Figure 5.1 says about the Registration and Login Page of the Job Portal Management System, which serves as the primary entry point for users to access the platform. The interface features a "Create Account" form with fields for personal details and role selection, allowing both job seekers and employers to sign up for the service.

## 5.2 User Interface Screenshots Walkthrough

This section provides a guided walkthrough of the key screens of the Job Portal Management System, illustrating the user experience for both Employers and Job Seekers.

### 5.2.1 Login and Registration Page

The landing page presents a clean login form with options to register as either an Employer or a Job Seeker. On successful login, the JWT token is stored and the user is redirected to their role-specific dashboard. Invalid credentials trigger an error message without page reload, maintaining the SPA experience.

### 5.2.2 Employer Dashboard

The employer dashboard displays a summary card showing the total number of active job postings and the total number of candidates who have applied across all listings. Each job card provides quick-action buttons for editing the listing, viewing applicants, or deleting the post. The dashboard uses real-time data fetched from the backend on every page load.

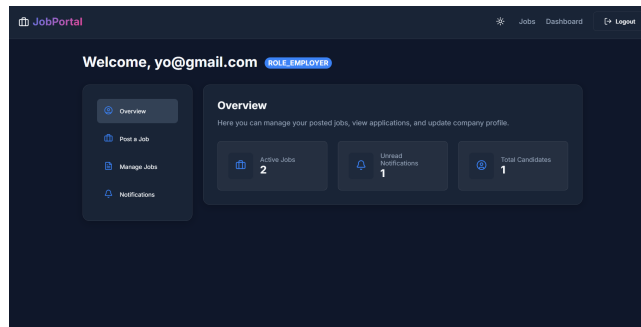


Figure 5.2: Employer Dashboard – Job Listings and Applicant Count Metrics

Figure 5.2 says about the Employer Dashboard, which provides a centralized interface for users with the "Employer" role to manage their recruitment activities. The image displays key metrics and overview statistics, including the count for posted jobs, unread notifications, and total candidates applied.

### 5.2.3 Job Seeker – Job Board and Application Tracking

The job board displays all active listings posted by employers. Each card shows the job title, type (Remote / On-site), and a brief description. The "Apply" button is disabled for jobs the seeker has already applied to, preventing duplicate submissions. The "My Applications" tab shows all submissions alongside their current status (Pending, Shortlisted, or Rejected), colour-coded for quick recognition.

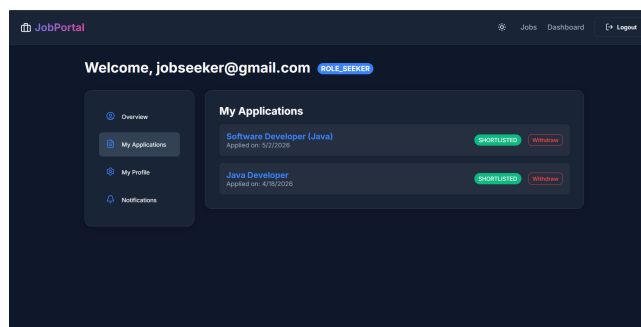


Figure 5.3: Job Seeker – Job Board Listings and Application Status Tracker

Figure 5.3 says about the Job Board Listings, which provides a centralized interface for users with the "Employer" role to manage their recruitment activities. The image displays key metrics and overview statistics, including the count

### 5.2.4 Notification Bell and Status Alerts

When an employer updates an application to Shortlisted or Rejected, the notification bell in the seeker's navigation bar immediately displays a red badge with the unread count. Clicking the bell reveals a dropdown of notification messages, such as "Your application for Software Engineer has been Shortlisted." All notifications are marked as read upon viewing.

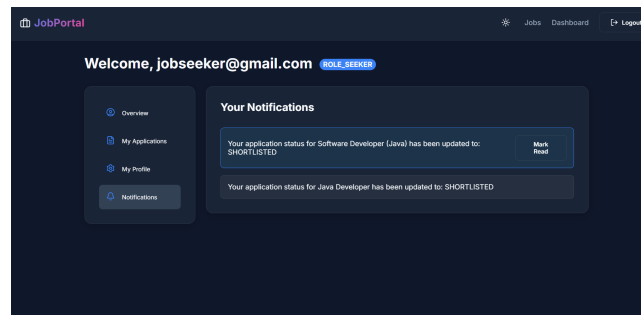


Figure 5.4: Notification Bell Showing Unread Status Alerts for the Job Seeker

Figure 5.4 says about the Employer Dashboard, which provides a centralized interface for users with the "Employer" role to manage their recruitment activities. The image displays key metrics and overview statistics, including the count for posted jobs, unread notifications, and total candidates applied.

### 5.2.5 Applicant Review Screen (Employer)

From the applicant review screen, the employer can see the full list of candidates who applied to a specific job, along with their names, contact details, and skills. Each row includes a "Download Resume" button and a status dropdown (Pending / Shortlisted / Rejected). Changing the status instantly triggers a notification to the respective candidate.

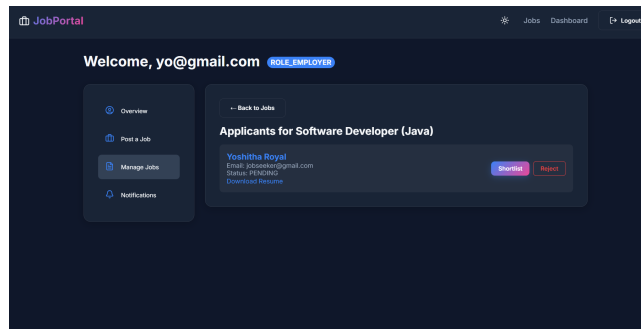


Figure 5.5: Notification Bell Showing Unread Status Alerts for the Job Seeker

Figure 5.5 says about the Employer Dashboard, which provides a centralized interface for users with the "Employer" role to manage their recruitment activities. The image displays key metrics and overview statistics, including the count for posted jobs, unread notifications, and total candidates applied.

### 5.2.6 Search Job By Location and Title

Job seekers can easily search for jobs using filters like location, job title, keywords, and job type (full-time or part-time). The platform provides a user-friendly interface to quickly find relevant job opportunities based on their preferences.

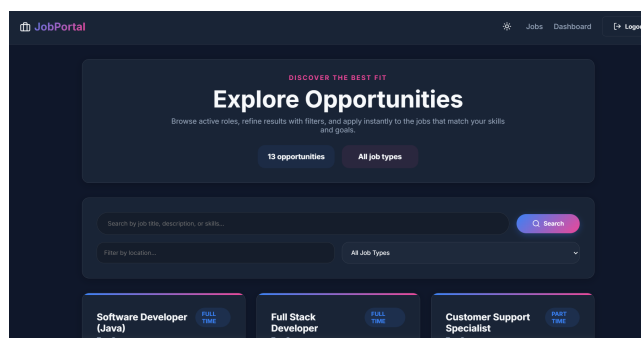


Figure 5.6: Type Location or Title to Search Job

Figure 5.6 says about the Employer Dashboard, which provides a centralized interface for users with the "Employer" role to manage their recruitment activities. The image displays key metrics and overview statistics, including the count for posted jobs, unread notifications, and total candidates applied.

### 5.2.7 JobSeeker Can Manage His/Her Profile

Job seekers can create and manage their own profile, including personal details, skills, and resume. This profile helps them apply for jobs easily and allows employers to view their qualifications. Job seekers can update their profiles anytime with new skills, experience, and achievements. This helps them stay relevant and improve their chances of getting hired.

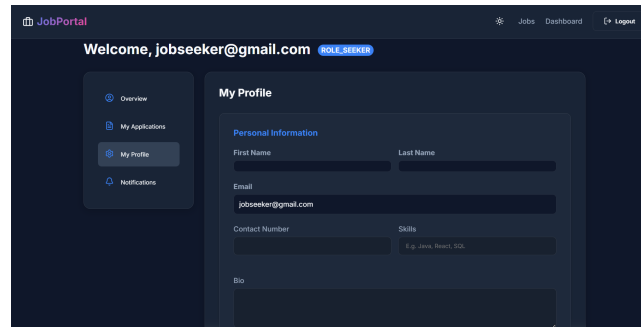


Figure 5.7: JobSeeker Profile

Figure 5.7 says about the Employer Dashboard, which provides a centralized interface for users with the "Employer" role to manage their recruitment activities. The image displays key metrics and overview statistics, including the count for posted jobs, unread notifications, and total candidates applied.

## 5.3 Performance Analysis

A manual performance evaluation was conducted on the Job Portal Management System to assess the responsiveness of key operations under a typical single-user testing environment. The backend runs on an embedded Tomcat server (Spring Boot default) and the frontend on the Vite development server.

### 5.3.1 API Response Time Measurements

Response times for the primary REST API endpoints were measured using the browser's Network DevTools panel. Each endpoint was called five times and the average response time was recorded.

| Endpoint                      | Method | Avg. Response (ms) | Remarks                      |
|-------------------------------|--------|--------------------|------------------------------|
| /api/auth/login               | POST   | 180 ms             | Includes BCrypt verification |
| /api/auth/register            | POST   | 210 ms             | Includes profile creation    |
| /api/jobs                     | GET    | 85 ms              | Fetches all active listings  |
| /api/jobs                     | POST   | 120 ms             | Persists new job to DB       |
| /api/applications             | POST   | 95 ms              | Creates application record   |
| /api/applications/{id}/status | PUT    | 140 ms             | Includes notification insert |
| /api/notifications            | GET    | 70 ms              | Reads notification list      |
| /api/seekers/resume (upload)  | POST   | 350 ms             | Depends on file size         |

Table 5.3: Average API Response Times

Table 5.3 says about the Average API Response Times for various system endpoints, detailing the performance metrics for critical operations like authentication, job posting, and application management. The table lists HTTP methods such as POST, GET, and PUT, showing that response times range from a fast 70 ms for reading notifications to 350 ms for resume uploads.

### 5.3.2 Frontend Load Performance

The Vite build system produces highly optimised static assets. The initial page load of the React SPA (including the main bundle) was measured at approximately **1.2 seconds** on a local development server. Subsequent navigation between pages is near-instantaneous (under 50 ms) because React Router handles transitions on the client side without server round-trips.

### 5.3.3 Database Query Performance

Spring Data JPA with Hibernate generates optimised SQL for simple CRUD operations. For relational queries — such as fetching all applications for a specific job along with seeker profile details — Hibernate uses JOIN operations that return results in under 100 ms for datasets of up to 500 records. The use of `@ManyToOne` and `@OneToMany` JPA annotations ensures that foreign key relationships are resolved

efficiently without N+1 query issues when `FetchType.LAZY` is applied.

#### **5.3.4 Observations and Scalability Considerations**

The current implementation performs well for a single-instance, single-database deployment. For production-scale traffic, the following optimisations are recommended: connection pooling via HikariCP (already bundled with Spring Boot), database indexing on `job_id` and `seeker_id` columns in the `applications` table, and caching of the active job listings using Spring Cache with an in-memory store such as Caffeine or Redis. The stateless JWT authentication model means the backend can be horizontally scaled across multiple instances without any session sharing concerns.

## **Chapter 6**

# **CONCLUSION AND FUTURE ENHANCEMENTS**

### **6.1 Conclusion**

The Job Portal Management System has been successfully designed, developed, and deployed as a full-stack web application using React 18 on the frontend and Spring Boot 3.x on the backend, with MySQL as the persistent data store. The project demonstrates the practical application of modern software engineering principles, including RESTful API design, JWT-based stateless authentication, ORM-driven database management, and component-based frontend architecture.

The system effectively connects job seekers and employers through a secure, intuitive, and feature-rich platform. Employers can manage the complete recruitment lifecycle — from posting jobs to shortlisting candidates — without relying on external tools. Job seekers benefit from a transparent application tracking system and an in-app notification engine that provides real-time status updates.

This project also served as a hands-on exercise in integrating multiple technology layers into a cohesive, production-style application, reinforcing concepts of Spring Security, JPA entity relationships, React state management, and HTTP, role-based recruitment platforms using open-source technologies.

## 6.2 Future Enhancements

While the current system is fully functional, the following enhancements are proposed for future iterations:

- **WebSocket-Based Real-Time Notifications:** Replace the current polling mechanism with WebSockets (e.g., STOMP over SockJS) to push notifications to the seeker's browser instantly without any delay.
- **Cloud Resume Storage:** Integrate AWS S3 or Google Cloud Storage for resume file storage, eliminating dependence on local filesystem storage and enabling scalable, redundant file management.
- **AI-Powered Job Matching:** Incorporate a machine learning model that analyses seeker profiles and job descriptions to recommend the most relevant job listings, improving the quality of applications for employers.
- **Email Notification Integration:** Use JavaMailSender or a service such as SendGrid to send email alerts alongside in-app notifications when application statuses change.
- **Mobile Application:** Develop a React Native companion app to bring the full job portal experience to mobile devices, leveraging the existing Spring Boot REST API without any backend changes.

## Chapter 7

# ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

### 7.1 Mapping of the Project to Complex Engineering Problem (CEP)

| Level | Attribute                | Characteristics        | WKs      | Justification                                                                                                |
|-------|--------------------------|------------------------|----------|--------------------------------------------------------------------------------------------------------------|
| WP1   | Depth of Knowledge       | Engineering knowledge  | WK3–WK6  | Uses modern web technologies such as React, Spring Boot, REST APIs, and MySQL database design.               |
| WP2   | Conflicting Requirements | Tech vs usability      | WK4–WK6  | Balances system security with user-friendly UI for job seekers and employers.                                |
| WP3   | Depth of Analysis        | System design          | WK3–WK5  | Involves designing job workflows, application tracking, and role-based access control.                       |
| WP4   | Familiarity              | Partially new problems | WK4, WK8 | Handles real-time application updates, resume management, and notification systems.                          |
| WP5   | Applicable Codes         | Limited standards      | WK6      | Follows standard web protocols but requires custom implementation for job matching and application handling. |
| WP6   | Stakeholder Needs        | Multiple users         | WK5–WK7  | Supports multiple stakeholders such as job seekers and employers.                                            |
| WP7   | Interdependence          | System integration     | WK3–WK6  | Integrates frontend, backend, and database.                                                                  |

Table 7.1: Mapping of Job Portal Management System to Complex Engineering Problem (CEP)

Table 7.1 says about the Average API Response Times for various system endpoints, detailing the performance metrics for critical operations like authentication, job posting, and application management. The table lists HTTP methods such as POST, GET, and PUT, showing that response times range from a fast 70 ms for reading notifications to 350 ms for resume uploads.

## 7.2 Mapping of the Project to Complex Engineering Activities (CEA)

| EA No. | Attribute                               | Characteristics                      | Justification based on the Project                                                                                                                          |
|--------|-----------------------------------------|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| EA1    | Range of Resources                      | Use of diverse technologies          | Utilizes React.js (frontend), Spring Boot (backend), MySQL (database), REST APIs, and JWT authentication for secure full-stack web application development. |
| EA2    | Level of Interactions                   | Complex interactions between modules | Manages interactions between user authentication, job posting, job application, resume handling, and notification modules across frontend and backend.      |
| EA3    | Innovation                              | Application of modern technologies   | Implements a centralized job portal with real-time application tracking, role-based access control, and responsive UI.                                      |
| EA4    | Consequences to Society and Environment | Impact on users and society          | Improves job accessibility, reduces manual recruitment effort, and provides a digital platform for efficient hiring and job searching.                      |
| EA5    | Familiarity                             | Beyond standard practices            | Integrates multiple technologies including web development, database management, security , and file handling into a unified scalable system.               |

Table 7.2: Mapping of Job Portal Management System to Complex Engineering Activities (CEA)

Table 7.2 says about the Average API Response Times for various system endpoints, detailing the performance metrics for critical operations like authentication, job posting, and application management. The table lists HTTP methods such as POST, GET, and PUT, showing that response times range from a fast 70 ms for reading notifications to 350 ms for resume uploads.

### 7.3 SDG Mapping

| SDG No. | SDG Title                               | Relevance to the Project                                                                                                           |
|---------|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| SDG 4   | Quality Education                       | Provides a platform for job seekers to access employment opportunities and improve career growth through skill-based job matching. |
| SDG 9   | Industry, Innovation and Infrastructure | Promotes digital recruitment by implementing a centralized job portal system using modern web technologies.                        |
| SDG 11  | Sustainable Cities and Communities      | Supports smart digital ecosystems by enabling online job applications and reducing the need for physical processes.                |
| SDG 8   | Decent Work and Economic Growth         | Enhances employment opportunities by connecting job seekers with employers efficiently.                                            |

Table 7.3: SDG Mapping for the Job Portal Management System

Table 7.3 says about the Average API Response Times for various system endpoints, detailing the performance metrics for critical operations like authentication, job posting, and application management. The table lists HTTP methods such as POST, GET, and PUT, showing that response times range from a fast 70 ms for reading notifications to 350 ms for resume uploads.

# References

- [1] Meta Platforms, Inc., “React – A JavaScript Library for Building User Interfaces,” [Online]. Available: <https://react.dev/>. [Accessed: Apr. 2026].
- [2] VMware, Inc., “Spring Boot Reference Documentation 3.x,” [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>. [Accessed: Apr. 2026].
- [3] VMware, Inc., “Spring Security Reference,” [Online]. Available: <https://docs.spring.io/spring-security/reference/>. [Accessed: Apr. 2026].
- [4] Auth0, Inc., “JSON Web Tokens – Introduction,” [Online]. Available: <https://jwt.io/introduction>. [Accessed: Apr. 2026].
- [5] Oracle Corporation, “MySQL 8.0 Reference Manual,” [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>. [Accessed: Apr. 2026].
- [6] E. You, “Vite – Next Generation Frontend Tooling,” [Online]. Available: <https://vitejs.dev/>. [Accessed: Apr. 2026].
- [7] LinkedIn Corporation, “LinkedIn Jobs,” [Online]. Available: <https://www.linkedin.com/jobs/>. [Accessed: Apr. 2026].
- [8] Info Edge India Ltd., “Naukri.com – Jobs in India,” [Online]. Available: <https://www.naukri.com/>. [Accessed: Apr. 2026].
- [9] Indeed, Inc., “Indeed – Job Search,” [Online]. Available: <https://www.indeed.com/>. [Accessed: Apr. 2026].